

Sesión 02
Líneas de Productos Software

Anass El Jabiry Kaddir

Departamento de Informática
Universidad de Almería

Almería, lunes 11 de noviembre de 2024

Índice de Contenidos

Página

1. Introducción	3
2. Desarrollo Paso a Paso	3
Paso 1: Creación del Proyecto GestionAmigos.....	3
Paso 2: Desarrollo de la Vista Mapa (VistaMapa.swift).....	4
Paso 3: Desarrollo de la Vista Imagen (VistaImagen.swift)	5
Paso 4: Desarrollo de la Vista Datos (VistaDatos.swift).....	6
Paso 5: Desarrollo de la Vista Detalle (VistaDetalle.swift)	7
Paso 6: Personalización y Ajuste de Vistas.....	8
3. Ejercicios Realizados	9
Ejercicio 1: Trailing Closures	9
Ejercicio 2: Implementación de la Función getMessage()	10
Ejercicio 3: Estructuras en Swift - Acceso Público vs. Privado.....	11
4. Conclusiones y Reflexiones	13

1. Introducción

El objetivo de esta segunda práctica fue continuar con el aprendizaje del lenguaje Swift y del entorno de desarrollo de Apple, Xcode, para implementar una aplicación más compleja que la anterior. En esta sesión desarrollamos una app de gestión de amigos, denominada **GestionAmigos**, que incluye una vista detalle compuesta por varias sub-vistas, tales como una vista de mapa, una imagen y datos adicionales. Además, nos centramos en la combinación de estas vistas, utilizando SwiftUI para organizar los componentes visuales y trabajar con diferentes frameworks.

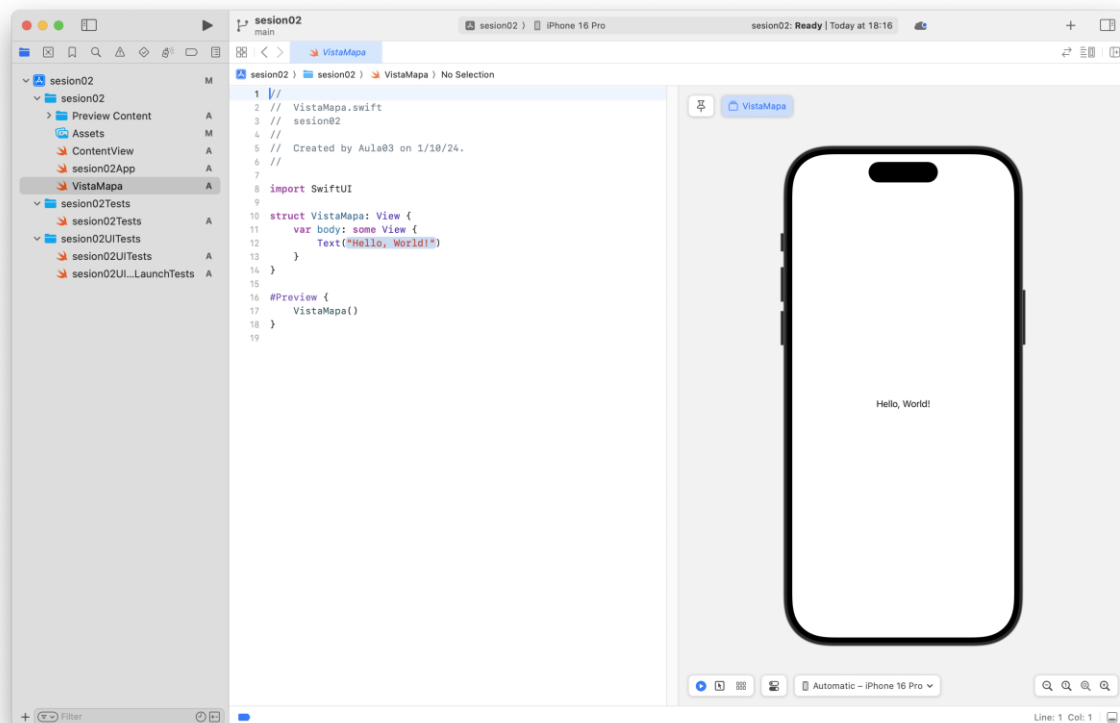
Herramientas Utilizadas

- **Xcode 13.4.1:** Utilizamos el entorno de desarrollo integrado (IDE) de Apple para el desarrollo de la app. Xcode permite gestionar el ciclo de desarrollo completo, incluyendo el diseño de la interfaz y la programación del código necesario.
- **Swift:** Continuamos con el uso del lenguaje Swift (versión 5), con el objetivo de aprender nuevas funcionalidades como trailing closures, estructuras y clases, y su aplicación en el desarrollo de aplicaciones.
- **SwiftUI:** Se utilizó nuevamente para la creación y combinación de vistas, aprovechando su poder para organizar componentes y crear una interfaz de usuario flexible.
- **MapKit:** Utilizado para implementar una vista de mapa dentro de nuestra aplicación, permitiendo centrar el mapa en unas coordenadas específicas.

2. Desarrollo Paso a Paso

Paso 1: Creación del Proyecto GestionAmigos

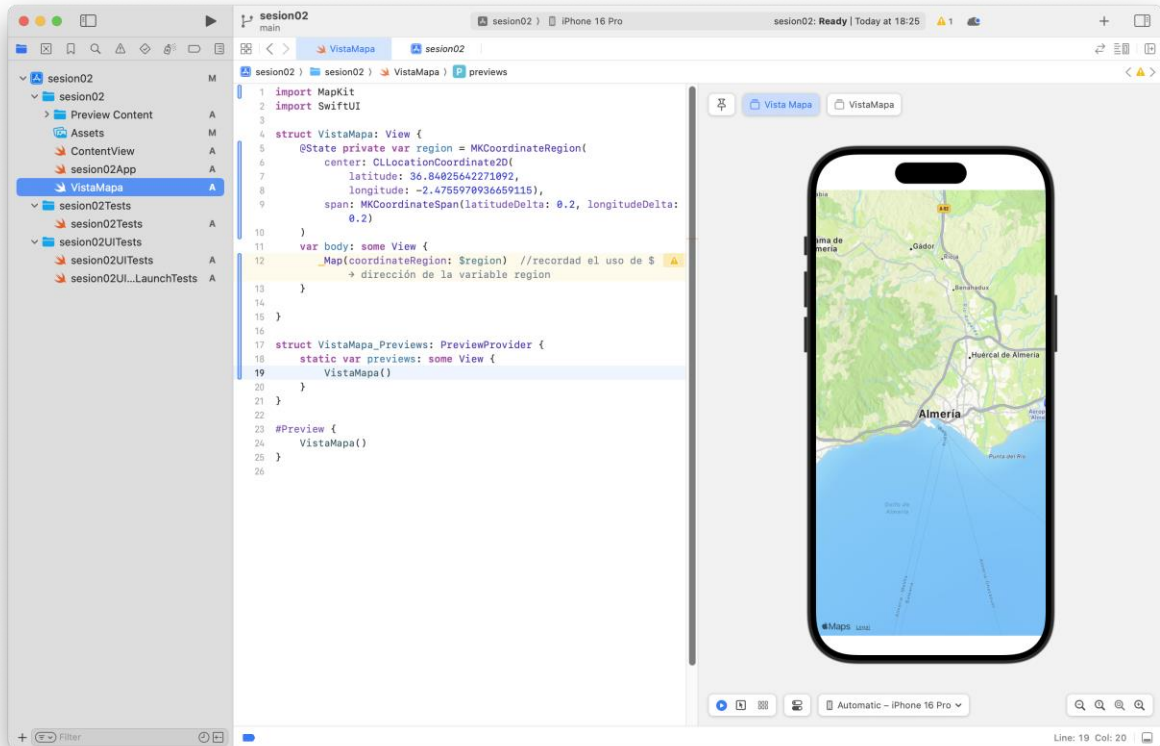
Se creó un nuevo proyecto en Xcode llamado **GestionAmigos** siguiendo la plantilla "iOS App". Elegimos Swift como lenguaje de programación y SwiftUI como el marco de trabajo para la interfaz de usuario.



Captura 1: Proyecto inicial en Xcode mostrando la estructura de archivos básica y el archivo principal.

Paso 2: Desarrollo de la Vista Mapa (VistaMapa.swift)

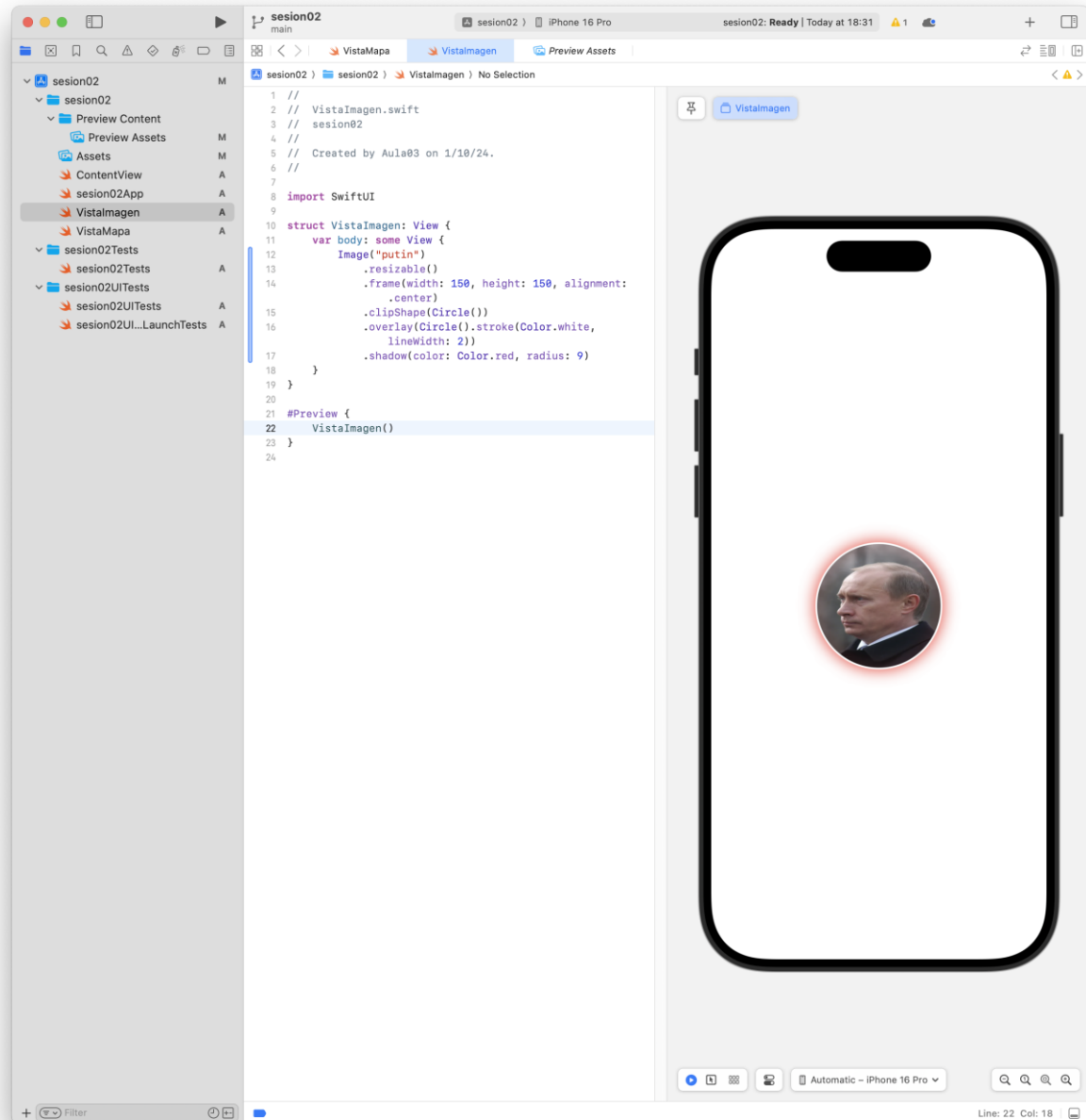
Creamos una vista llamada **VistaMapa** en la que se mostraba un mapa centrado en unas coordenadas definidas. Para ello utilizamos MapKit, importándolo al archivo VistaMapa.swift y configurando una variable de estado llamada *region*, la cual se vinculó al componente de tipo Map de SwiftUI.



Captura 2: VistaMapa mostrando la localización establecida con el marco de MapKit.

Paso 3: Desarrollo de la Vista Imagen (VistaImagen.swift)

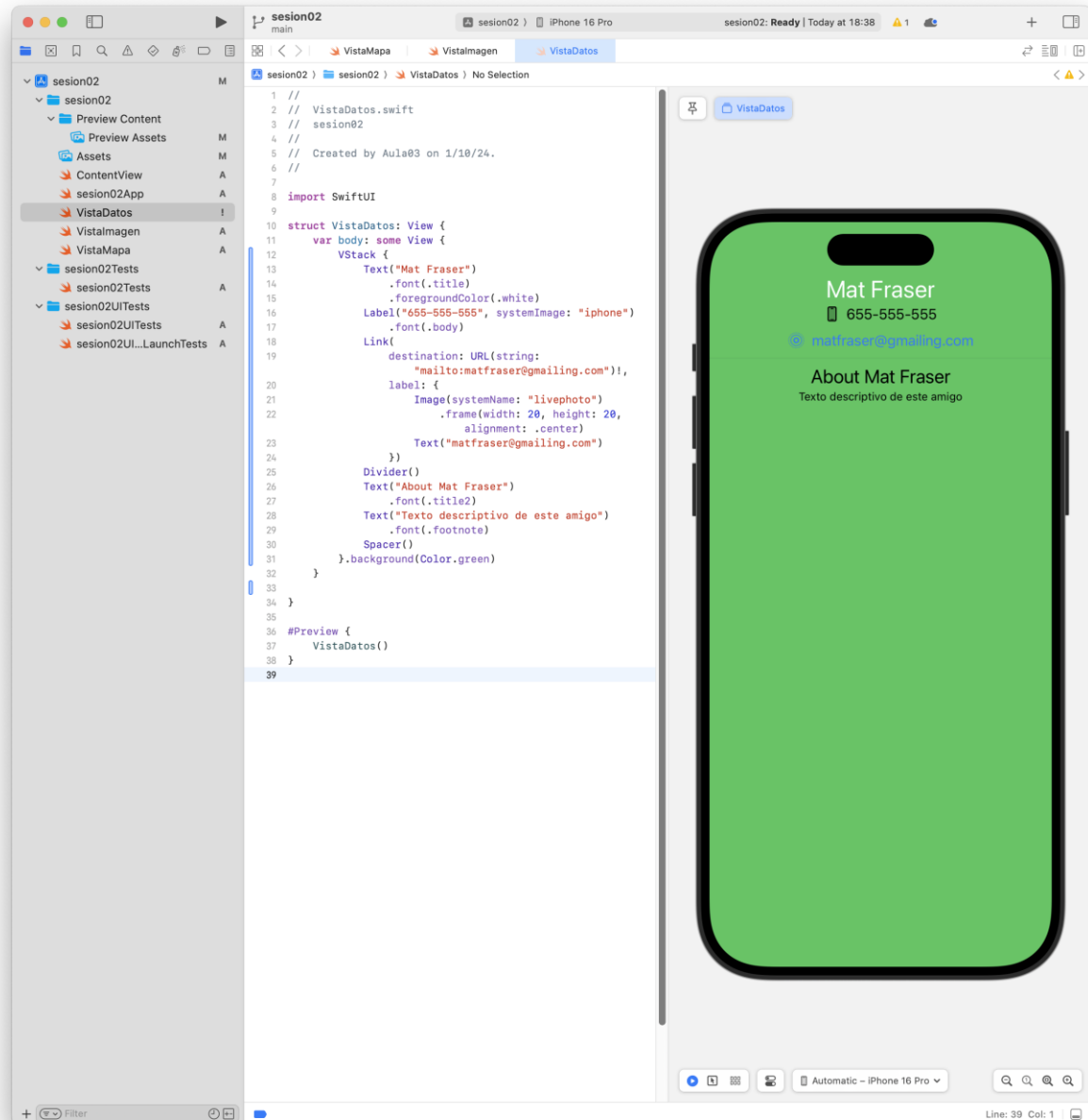
Se creó otra vista llamada **VistaImagen** que incluía una imagen circular. Se utilizó el componente Image para mostrar una fotografía descargada previamente y agregada al proyecto a través de Assets.xcassets. Se aplicaron modificadores como frame, clipShape, overlay y shadow para mejorar la apariencia visual.



Captura 3: VistaImagen mostrando la imagen de Mat Fraser, con forma circular, borde blanco y sombra roja.

Paso 4: Desarrollo de la Vista Datos (VistaDatos.swift)

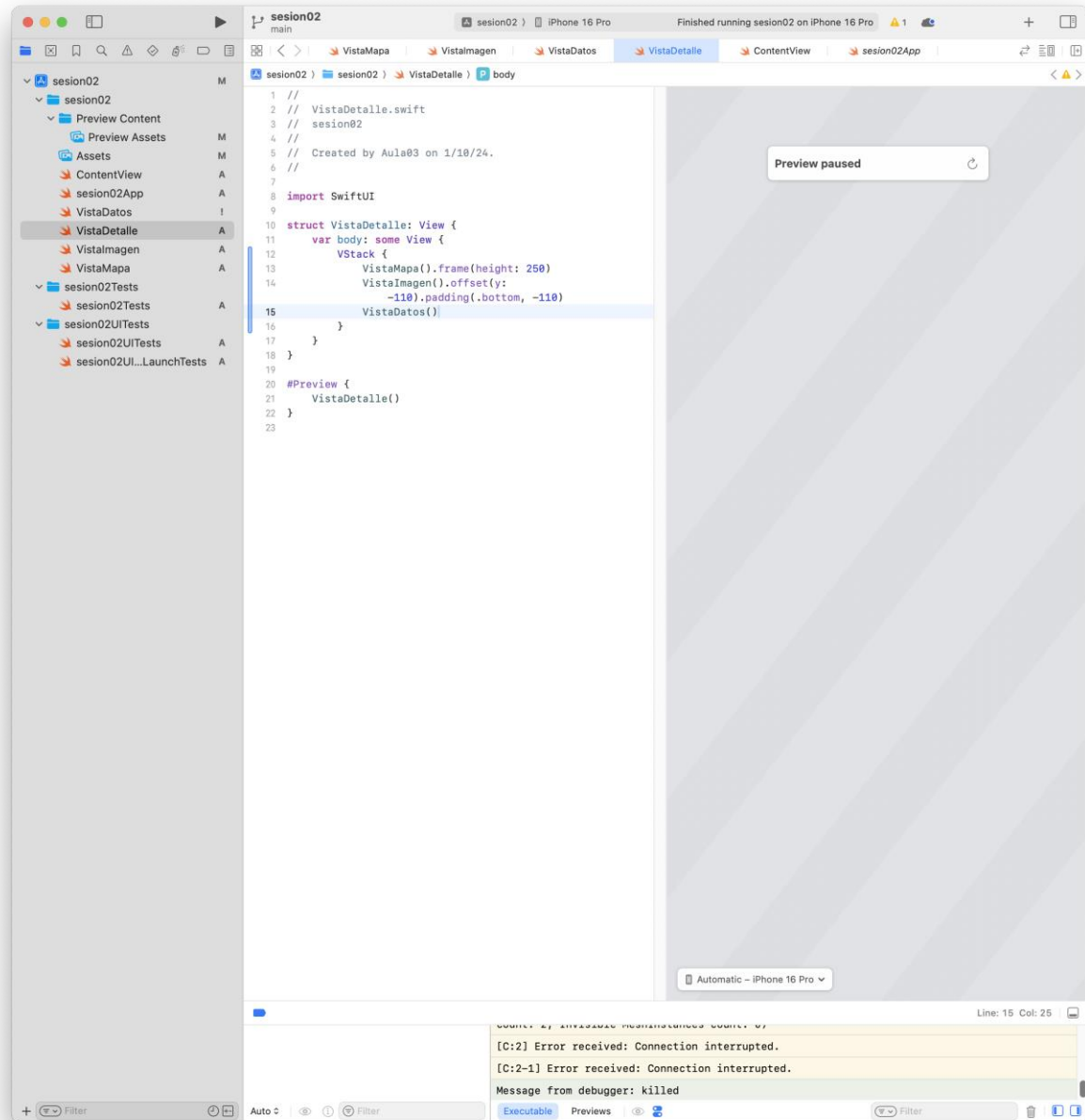
La vista de datos incluía información adicional sobre el amigo (o persona) con dos símbolos SF (San Francisco Symbols) para representar el teléfono y el correo electrónico. Se creó un VStack que contenía componentes de texto y otros elementos visuales, como Label y Link, para representar la información de contacto.



Captura 4: VistaDatos mostrando la información del contacto, incluyendo un link con un icono representativo.

Paso 5: Desarrollo de la Vista Detalle (VistaDetalle.swift)

La vista detalle, denominada **VistaDetalle**, combinaba las vistas desarrolladas anteriormente: **VistaMapa**, **VistaImagen**, y **VistaDatos**, organizándolas dentro de un **VStack** para crear una presentación coherente. Se realizaron ajustes visuales con modificadores como **frame**, **padding**, y **offset** para mejorar la composición.



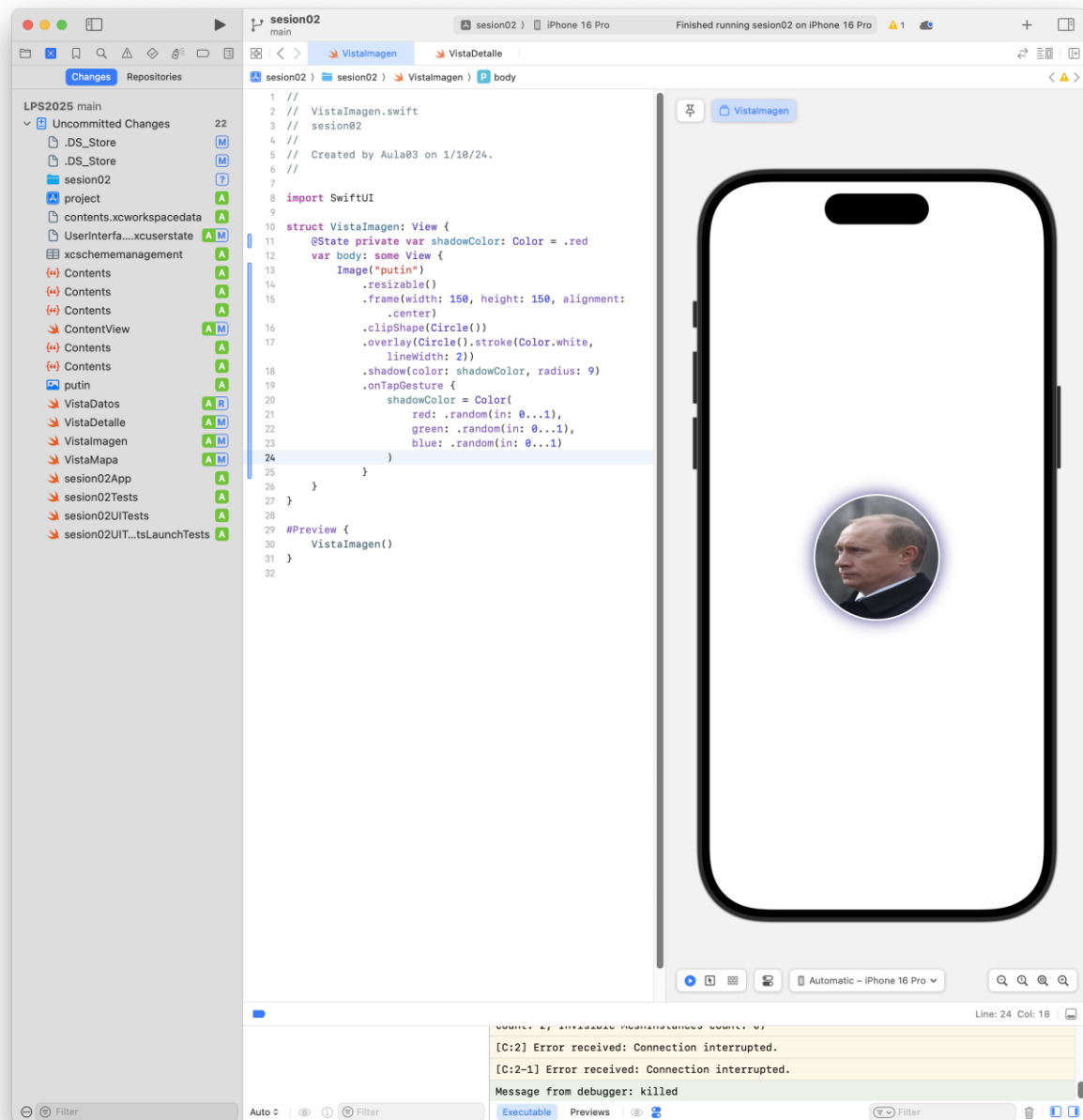
Captura 5: VistaDetalle mostrando la combinación de todas las sub-vistas, presentadas de manera organizada.

Paso 6: Personalización y Ajuste de Vistas

Se probaron diferentes personalizaciones en la combinación de vistas:

- Se ajustó el frame de la vista del mapa.
- La vista de la imagen se desplazó utilizando offset para superponerla al mapa.

También se implementó una interacción sencilla: al hacer "tap" sobre la imagen, se cambiaba el color de la sombra y el grosor de la línea, utilizando el modificador `.onTapGesture{}` junto con una variable de estado para controlar el comportamiento.



Captura 6: Interacción con VistaImagen, mostrando el cambio de color de la sombra tras la acción de "tap".

3. Ejercicios Realizados

Ejercicio 1: Trailing Closures

Practicamos la implementación de funciones que toman clausuras como último parámetro utilizando la sintaxis conocida como "trailing closure". Esto permite simplificar las llamadas a funciones y mejorar la legibilidad del código.

```

3  func functionEjercicio01( valor: Int, accion1: (Int) -> String, accion2: (Int) -> String ) {
4      , \
5      for i in 1...valor {
6          print("Iteración \(i)", "Acción 1", accion1(i), separator: " -> ")
7          print("Iteración \(i)", "Acción 2", accion2(i), separator: " -> ")
8      }
9  }
10 }
11
12 functionEjercicio01(
13     valor: 5,
14     accion1: { numero in
15         if numero % 2 != 0 {
16             return "Numero \(numero) es impar"
17         } else {
18             return "Numero \(numero) no es impar"
19         }
20     },
21     accion2: { numero in
22         if numero % 2 == 0 {
23             return "Numero \(numero) es par"
24         } else {
25             return "Numero \(numero) no es par"
26         }
27     }
28 )
29
30 functionEjercicio01(valor: 5) {
31     numero in
32     if numero % 2 != 0 {
33         return "Numero \(numero) es impar"
34     } else {
35         return "Numero \(numero) no es impar"
36     }
37 } accion2: { numero in
38     if numero % 2 == 0 {
39         return "Numero \(numero) es par"
40     } else {
41         return "Numero \(numero) no es par"
42     }
43 }

```

Output:

- Numero 5 es impar
- Numero 4 no es impar
- Numero 4 es par
- Numero 5 no es par
- Numero 5 es impar
- Numero 4 no es impar
- Numero 4 es par
- Numero 5 no es par

Line: 44 Col: 1

- **Ejemplo:** Se implementó la función `funcionEspecial()` que acepta un bloque como parámetro y se llamó a esta función utilizando diferentes sintaxis: normal, trailing closure, y con el último parámetro omitido.

Ejercicio 2: Implementación de la Función getMessage()

Implementamos la función `getMessage(lugar: String, accion: (Int, String) -> String) -> String` que toma un lugar como parámetro y una clausura que describe el tipo de viaje. Esta función se diseñó para generar diferentes mensajes dependiendo del tipo de viaje seleccionado aleatoriamente.

```

1 import UIKit
2
3 func getMessage(lugar: String, accion: (Int, String) -> String) -> String {
4     return accion(Int.random(in: 1...3), lugar)
5 }
6
7 var mensaje: String = getMessage(lugar: "Londres") {
8     (tipoViaje, lugar) -> String in
9     switch tipoViaje {
10     case 1:
11         return "me voy de marcha a \(lugar)"
12     case 2:
13         return "me voy a trabajar a \(lugar)"
14     case 3:
15         return "me voy a estudiar a \(lugar)"
16     default:
17         return "me voy a \(lugar)"
18     }
19 }
20
21 print(mensaje)
22
23 mensaje = getMessage(lugar: "Londres") {
24     switch $0 {
25     case 1:
26         return "me voy de marcha a \($1)"
27     case 2:
28         return "me voy a trabajar a \($1)"
29     case 3:
30         return "me voy a estudiar a \($1)"
31     default:
32         return "me voy a \($1)"
33     }
34 }
35
36 print(mensaje)
37

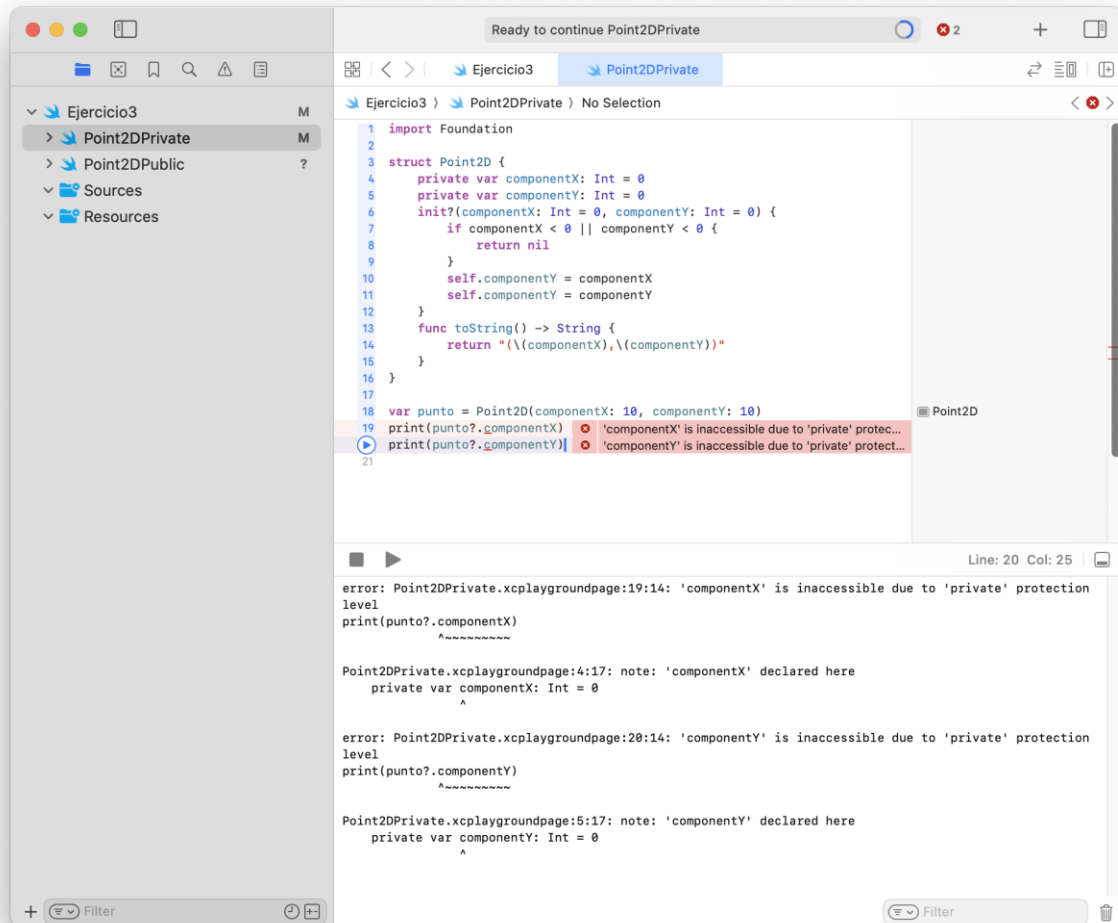
```

me voy de marcha a Londres
me voy de marcha a Londres

Ejercicio 3: Estructuras en Swift - Acceso Público vs. Privado

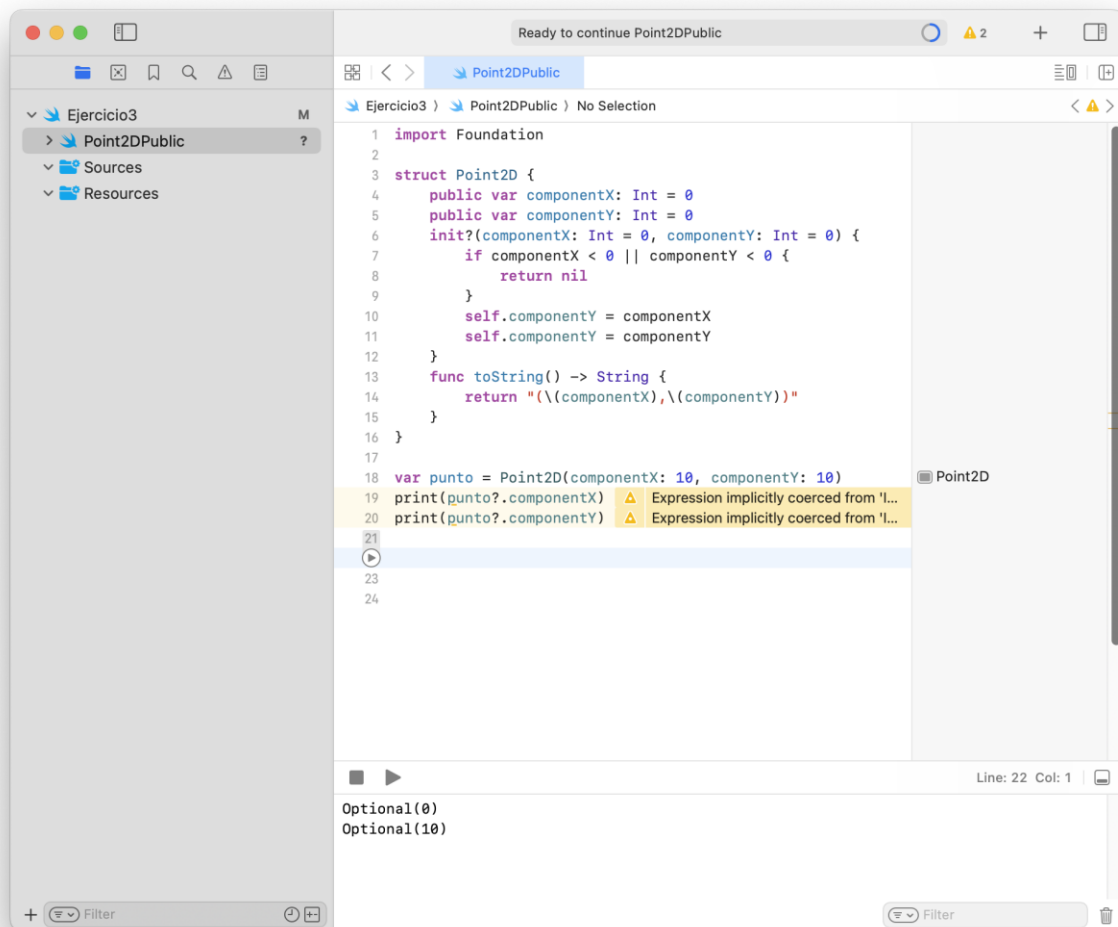
Exploramos cómo el nivel de acceso (public vs private) afecta la accesibilidad de los atributos de una estructura en Swift.

- **Código con acceso private:**



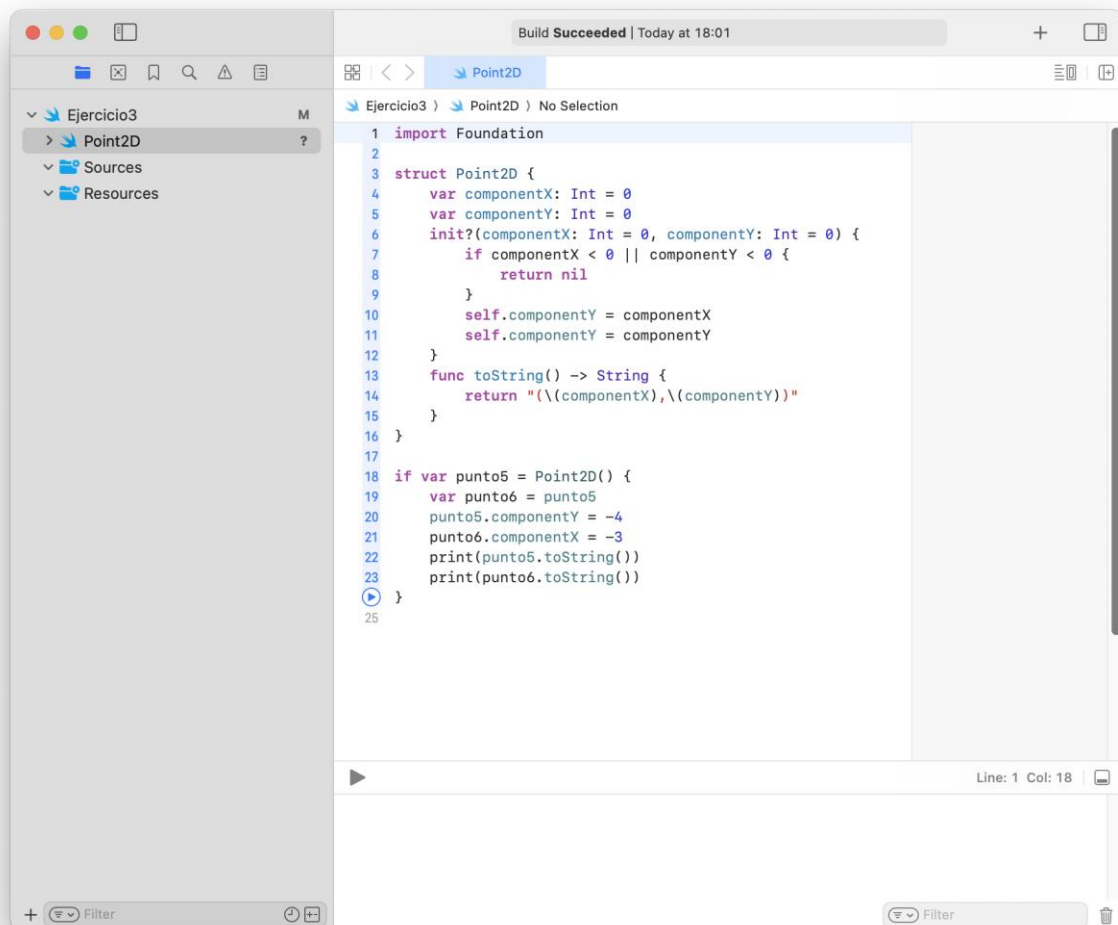
En este caso, los atributos `componentX` y `componentY` son `private`, por lo que no es posible acceder a ellos directamente desde fuera de la estructura, como se muestra en los errores del código.

- **Código con acceso public:**



Al cambiar el acceso de private a public, los atributos componentX y componentY son accesibles desde fuera de la estructura.

- **Ejemplo con Copia de Estructuras:**



En este ejemplo, se observa que punto5 y punto6 son copias independientes debido a que Point2D es una estructura (valor por copia). Al modificar punto5 o punto6, los cambios no afectan a la otra variable.

- **NOTA:** Si Point2D fuera una clase, punto5 y punto6 serían referencias al mismo objeto, y modificar una afectaría a la otra, ya que en Swift, las clases son manejadas por referencia (gestión dinámica en heap), mientras que las estructuras son manejadas por valor (gestión estática en stack).

4. Conclusiones y Reflexiones

En esta sesión aprendimos a combinar diferentes vistas para crear una aplicación más completa y funcional. También profundizamos en el uso de nuevas funcionalidades del lenguaje Swift, como las trailing closures, y en el uso de estructuras y clases. La práctica nos permitió entender cómo utilizar distintos frameworks dentro de una misma aplicación y cómo organizar código para mantenerlo modular y legible.

El desarrollo de **GestionAmigos** fue un paso significativo hacia la construcción de aplicaciones más complejas y la utilización de buenas prácticas para el diseño de interfaces y la gestión de estados.